MPOO 2027-1 · GRUPO 06

Parcial 1 · correcciones

Versión D del examen del 7 de octubre. Para cada pregunta vienen el enunciado, tu respuesta tal como la enviaste en el portal y su corrección. La calificación no va aquí: la ves con el profesor.

Antes de empezar. Este archivo lo subió el profesor a tu rama. Trae el cambio a tu computadora antes de tu siguiente push, o tu push va a fallar:

```
git checkout entregas_villarruel_david
git pull --no-rebase
```

Qué tienes que hacer. En hojas, **a mano**, copia cada pregunta y escribe debajo tu respuesta corregida, usando la corrección que viene aquí. Donde la corrección ya te da la salida o el código, no basta con copiarlo: escribe **por qué** sale así, con la tabla de seguimiento de las variables, la causa del error o lo que devuelve tu método en cada renglón de la tabla. Debajo de esas preguntas viene qué escribir. Escanea las hojas o tómales foto, júntalas en un PDF y súbelo a esta misma carpeta, `entregas/villarruel_david/examen/p1/`, con tu push de siempre.

Pregunta 0.1 · Preguntas abiertas

Pregunta

Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de código
¿Qué hace new al crear un objeto? Explica qué recibe la variable a la que asignas esa expresión.

Tu respuesta

```
Instanciar a un objeto a partir de la función constructora o una clase.
```

> **Corrección.** Correcta pero incompleta. Dice bien que new crea una instancia a partir de la clase y usa el constructor, así que tiene la parte de creación e inicialización. Falta la segunda parte de la pregunta: la expresión new produce una referencia al objeto creado, y eso es lo que guarda la variable.

Pregunta 0.2 · Preguntas abiertas

Pregunta

¿Qué significa void en la cabecera de un método? Explica si ese método puede modificar un atributo.

Tu respuesta

Sin respuesta.

> **Corrección.** Sin respuesta.

Pregunta 0.3 · Preguntas abiertas

Pregunta

¿Para qué sirve this? Explica cómo distingue un atributo de un parámetro que tiene el mismo nombre.

Tu respuesta

Sin respuesta.

> **Corrección.** Sin respuesta.

Pregunta 0.4 · Preguntas abiertas

Pregunta

Java siempre pasa por valor. ¿Qué recibe un método cuando le pasas un int y cuando le pasas un arreglo?

Tu respuesta

Sin respuesta.

> **Corrección.** Sin respuesta.

Pregunta 0.5 · Preguntas abiertas

Pregunta

¿En qué se distinguen JDK y JRE? Explica cuál necesitas para compilar un programa y por qué.

Tu respuesta

Sin respuesta.

> **Corrección.** Sin respuesta.

R1 · ¿Qué imprime?

Pregunta

Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido.

```
Integer base = 5;
Integer copia = base;
int[] aumentos = {1, 2, 3};
for (int aumento : aumentos) {
    base = base + aumento;
    System.out.println(base / 2);
}
String etiqueta = "D" + copia + 1;
System.out.println(etiqueta);
System.out.println((double) (base / 2));
```

Tu respuesta

```
3
4
5
5.0
```

> **Corrección.** Las líneas 3, 4, 5 y 5.0 son correctas y están en orden. Falta la cuarta línea, D51: "D" + copia + 1 concatena de izquierda a derecha, y copia sigue valiendo 5 aunque base cambie.

Qué escribir a mano. La salida ya la tienes en la corrección. Escribe por qué sale así: una tabla de seguimiento con el valor de cada variable en cada vuelta y en cada println.

R2 a) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
if (true) { int voltaje = 12; }
System.out.println(voltaje);
```

Tu respuesta

Clasificación

```
No compila
```

Causa o salida

```
se declara dentro del if
```

Corrección. Correcto: no compila porque voltaje se declara dentro del bloque del if y fuera de él no existe, aunque la condición sea true.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 b) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
StringBuilder ruta = new StringBuilder("Q");
ruta.append(2 + 3);
System.out.println(ruta);
```

Tu respuesta

Clasificación

```
Funciona
```

Causa o salida

```
Qs
```

Corrección. La clasificación 'Funciona' es correcta. La salida no lo es: imprime exactamente Q5, porque primero se evalúa 2 + 3 = 5 y luego append añade ese 5 al StringBuilder. 'Qs' no es la salida completa, y tampoco explicó la regla.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 c) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
int[] canales = null;
System.out.println(canales.length);
```

Tu respuesta

Clasificación

```
Compila pero truena
```

Causa o salida

Sin respuesta.

> **Corrección.** La clasificación "compila pero truena" es correcta. No dio la causa: canales es null y leer su length lanza NullPointerException antes de imprimir nada.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R3 · Modelado: base, reto y justificación

Pregunta

Cada termostato guarda etiqueta fija y temperatura. El rango común del edificio es de 16 a 28 grados. Constructor y ajustar(int temperatura) reciben valores válidos; ajustar asigna. Añade una consulta de temperatura.

Base. Declara al menos dos atributos de instancia con tipo, uno static y una constante final (¿también static?). Completa constructor y dos métodos: cambio y consulta.

Reto. ¿Cómo declaras los dos extremos del rango para que sean comunes y no cambien?

Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

Tu respuesta

Clase

Sin respuesta.

Justificaciones

Sin respuesta.

> **Corrección.** Sin respuesta.

R4 · Código con repetición

Pregunta

Devuelve el promedio entero de los valores pares de pulsos usando for-each. Si no hay pares, devuelve 0. El arreglo puede estar vacío, no es null y admite negativos. La suma cabe en int. No modifiques el arreglo.

```
static int promedioPares(int[] pulsos)
```

Ejemplos (argumentos → retorno):

{3, 8, 2, 5} → 5 · {1, 3, 5} → 0 · {-4, 3, 6} → 1

Escribe el método completo. No imprimas ni leas del teclado.

Tu respuesta

Sin respuesta.

> **Corrección.** Sin respuesta.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.

R5 · Código con selección

Pregunta

Clasifica la temperatura entera de una cámara de pruebas. Usa if / else-if / else y devuelve el texto de la tabla. Los límites están incluidos. No imprimas ni leas del teclado.

```
static String estadoTermico(int temperatura)
```

Regla o valor recibido	Texto devuelto
Menor que 0 o mayor que 120	"INVALIDA"
0 a 29	"FRIA"
30 a 59	"TEMPLADA"
60 a 89	"CALIENTE"
90 a 120	"MUY CALIENTE"

Tu respuesta

Sin respuesta.

> **Corrección.** Sin respuesta.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.